

Contents

Zeus16 CPU Emulator	2
Team 3 :	2
1. System Architecture & Code Structure	2
1.1 System Architecture	2
1.2 ISA	3
1.3 Code structure	4
2. Verification	5
2.1 main program	5
2.2 Edge case verification Program	7

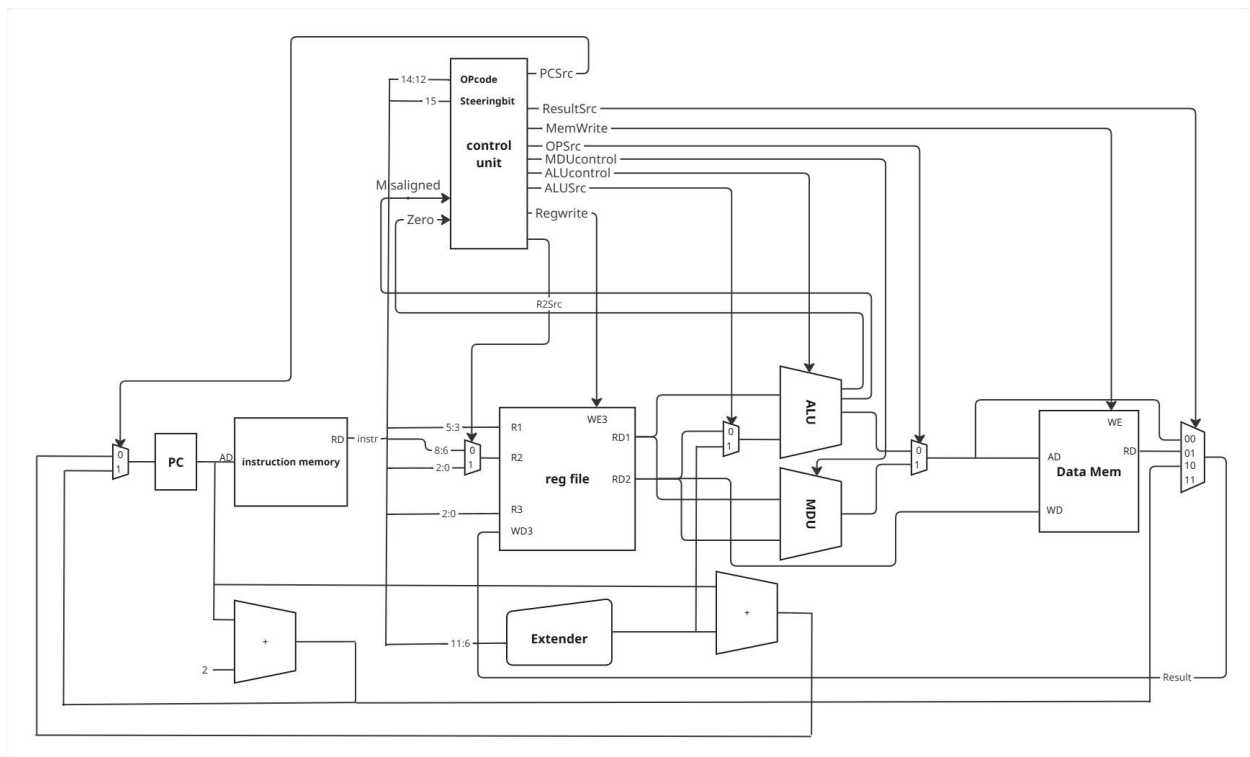
Zeus16 CPU Emulator

Team 3 :

Member #	Student Name	Role / Responsibilities
Member 1	Abdallah Reda	-Architecture -deassembler
Member 2	Mahmoud Gamal	-ISA - decoder – program machine code
Member 3	Mina Ayad	- Code structure – Memory – documentation
Member 4	Eman Yasser	-main -wrapper class CPU
Member 5	Mennatullah Osama	- ALU - verification plan

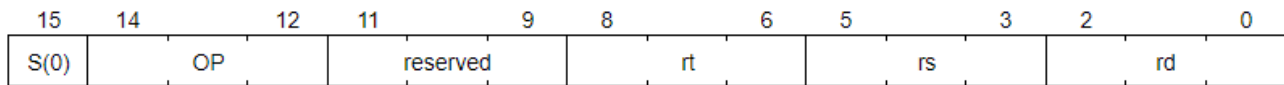
1. System Architecture & Code Structure

1.1 System Architecture

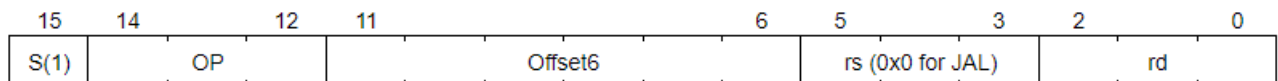


1.2 ISA

1.2.1 Register Instructions format



1.2.2 I & J Instructions format



1.2.3 Opcodes

Register Instructions		Immediate & Jump instructions	
Steering bit	0	Steering bit	1
Add	000	Addi	000
Sub	001	Andi	001
And	010	Ori	010
Or	011	Slti	011
SLT	100	Lw	100
MUL	101	Sw	101
Div	110	Beq	110
Rem	111	Jal	111

1.3 Code structure

FILE NAME	ROLE / COMPONENT	KEY RESPONSIBILITIES & DESCRIPTION
INSTRUCTION.H	Decoded Struct & Control Signals	A container/struct that holds all information about an instruction after decoding (register numbers, immediate numbers, and control flags).
INSTRUCTIONMEMORY.H	I-Mem / ROM & Hex Parser	Stores the 64KB program code. It reads the hex machine code from the .hex file and gives instructions to the CPU one by one.
DECODER.H	Control Unit & Bit Field Parser	Splits the 16-bit binary code into opcode, register numbers, and immediate values. It does sign extension for negative numbers and turns on control flags.
DISASSEMBLER.H	Disassembler & Real-Time Logger	Convert fetched binary instruction into assembly code
ALU.H	Arithmetic Logic Unit (Base I + M)	Performs all math and logic: Add, Subtract, Multiply, Divide, AND, OR, and Comparisons. It also safely handles Divide-by-0 without crashing
DATAMEMORY.H	D-Mem / RAM & Strict Alignment	Stores variables (64KB memory "byte addressable ") for Load (LW) and Store (SW). It automatically stops the CPU with halted flag if an invalid odd address is accessed.
CPU.H	Top-Level CPU Wrapper & Pipeline Controller	The main brain that connects all components together. It manages the 8 registers (R0–R7), tracks the Program Counter (PC), and runs the fetch-decode-execute loop.
MAIN.CPP	Testbench & Simulator Entry Point	The starting point that launches the CPU, loads the .hex files, runs the execution steps, and prints the registers and memory.

2. Verification

2.1 main program

2.1.1 objective

The program inside program.hex is the main benchmark test. Its main target is to test every instruction type in our CPU design in one complete program:

- Math Operations: Tests Addition (ADD, ADDI), Subtraction (SUB), Multiplication (MUL), Division (DIV), and Remainder (REM).
- Logic Operations: AND (AND, ANDI) and Bitwise OR (OR, ORI).
- Comparisons: Set Less Than (SLT, SLTI).
- Memory Access: Tests writing a value to RAM (SW) and loading it back (LW).
- Control Flow: backward looping with Branch on Equal (BEQ) and forward jumping with JAL (skipping instructions in between).

PC	Hex	Instruction	Expected Result	Execution Status
0x0000	0x8141	ADDI R1, R0, 5	R1 = 5	Executed
0x0002	0x8542	ADDI R2, R0, 21	R2 = 21	Executed
0x0004	0x008B	ADD R3, R1, R2	R3 = 26	Executed
0x0006	0x8084	ADDI R4, R0, 2	R4 = 2	Executed
0x0008	0x511D	MUL R5, R3, R4	R5 = 52	Executed
0x000A	0x408A	SLT R2, R1, R2	R2 = 1 (True)	Executed
0x000C	0xB1A6	SLTI R6, R4, 6	R6 = 1 (True)	Executed
0x000E	0x10AF	SUB R7, R5, R2	R7 = 51	Executed
0x0010	0x308E	OR R6, R1, R2	R6 = 5	Executed
0x0012	0xD005	SW R5, 0(R0)	Mem[0] = 52	Executed
0x0014	0x605D	DIV R5, R3, R1	R5 = 5	Executed
0x0016	0xC005	LW R5, 0(R0)	R5 = 52 (Restored)	Executed
0x0018	0x208E	AND R6, R1, R2	R6 = 1	Executed
0x001A	0xA236	ORI R6, R6, 8	R6 = 9	Executed
0x001C	0x93B6	ANDI R6, R6, 14	R6 = 8	Executed
0x001E	0x807F	ADDI R7, R7, 1	1: R7=52 2: R7=53	Executed (2x)
0x0020	0xEFBD	BEQ R5, R7, -2	1: Branch taken 2: Exit	Executed (2x)
0x0022	0xF200	JAL R0, 8	PC = 0x0022 + 8 = 0x002A	Executed (Jump)
0x0024	0xD005	SW R5, 0(R0)	IGNORED (Skipped by JAL)	Skipped
0x0026	0x611D	DIV R5, R3, R4	IGNORED (Skipped by JAL)	Skipped
0x0028	0xC005	LW R5, 0(R0)	IGNORED (Skipped by JAL)	Skipped
0x002A	0x706E	REM R6, R5, R1	R6 = 2 (Target Landed)	Executed (Target)

REGISTER	R0	R1	R2	R3	R4	R5	R6	R7
VALUE	0	0x5 (5)	0x1 (1)	0x1A (26)	0x2 (2)	0x34 (52)	0x2 (2)	0x35 (53)
MEMORY (ADDRESS)	0x0000							
VALUE	0x34(52)							

2.1.2 Code result

```

=====
CPU Instance 1: Normal Execution (program.hex)
=====
Running program...
----- Disassembled Program -----
0x0000: ADDI R1, R0, 5
0x0002: ADDI R2, R0, 21
0x0004: ADD R3, R1, R2
0x0006: ADDI R4, R0, 2
0x0008: MUL R5, R3, R4
0x000a: SLT R2, R1, R2
0x000c: SLTI R6, R4, 6
0x000e: SUB R7, R5, R2
0x0010: OR R6, R1, R2
0x0012: SW R5, 0(R0)
0x0014: DIV R5, R3, R1
0x0016: LW R5, 0(R0)
0x0018: AND R6, R1, R2
0x001a: ORI R6, R6, 8
0x001c: ANDI R6, R6, 14
0x001e: ADDI R7, R7, 1
0x0020: BEQ R5, R7, -2
0x001e: ADDI R7, R7, 1
0x0020: BEQ R5, R7, -2
0x0022: JAL R0, 8
0x002a: AND R6, R1, R2
----- CPU Registers -----
R0 = 0x0000 (0)
R1 = 0x0005 (5)
R2 = 0x0001 (1)
R3 = 0x001a (26)
R4 = 0x0002 (2)
R5 = 0x0034 (52)
R6 = 0x0001 (1)
R7 = 0x0035 (53)
PC = 0x0036
HALTED = false

----- Instruction Memory -----
address | instruction
0x00    | 0x8141
0x02    | 0x8542
0x04    | 0x8b
0x06    | 0x8084
0x08    | 0x511d
0x0a    | 0x408a
0x0c    | 0xb1a6
0x0e    | 0x10af
0x10    | 0x308e
0x12    | 0xd005
0x14    | 0x605d
0x16    | 0xc005
0x18    | 0x208e
0x1a    | 0xa236
0x1c    | 0x93b6
0x1e    | 0x807f
0x20    | 0xefaf
0x22    | 0xf200
0x24    | 0xd005
0x26    | 0x611d
0x28    | 0xc005
0x2a    | 0x208e

----- Data Memory -----
address | data
0x0000  | 0x34

```

2.2 Edge case verification Program

2.2.1 objective

The program inside program_halt.hex is dedicated for edge-case verification test.

- Divide-by-Zero Protection: dividing by zero returns 0xFFFF (-1) without crashing.
- Remainder-by-Zero Protection: modulo by zero safely returns the original dividend number a.
- JAL Return Address Linking: JAL saves the return address (PC + 2) into a register when rd is not R0.
- JAL Instruction Skipping: JAL skips intermediate dummy instructions and lands on the target.
- Signed Immediate Comparison: Verifies signed comparison with SLTI ($0 < 5 = 1$).
- Strict Alignment Trap: Verifies that attempting to write to an odd unaligned address (0x0011) triggers a hardware alignment trap, sets HALTED = true, and stops execution immediately.
- Unreached Code Protection: Verifies that instructions placed after the alignment trap are never executed.

PC	Hex	Instruction	Expected Result	Execution Status
0x00	0x8641	ADDI R1, R0, 25	R1 = 25	Executed
0x02	0x600A	DIV R2, R1, R0	R2 = 0xFFFF (-1)	Executed (DIV/0)
0x04	0x700B	REM R3, R1, R0	R3 = 25 (operand a)	Executed (REM/0)
0x06	0xF104	JAL R4, 4	R4 = 8, PC = 0x000A	Executed (Jump)
0x08	0x88C5	ADDI R5, R0, 99	IGNORED (Skipped by JAL)	SKIPPED
0x0A	0xB145	SLTI R5, R0, 5	R5 = 1 (Target Landed)	Executed (Target)
0x0C	0x8406	ADDI R6, R0, 16	R6 = 16	Executed
0x0E	0xD031	SW R1, 0(R6)	Mem[16] = 25 (Succeeds)	Executed (Valid)
0x10	0xD071	SW R1, 1(R6)	Alignment Trap! HALTED = true	TRAPPED & HALTED
0x12	0x8007	ADDI R7, R0, 77	UNREACHED (R7 stays 0)	UNREACHED

REGISTER	R0	R1	R2	R3	R4	R5	R6	R7
VALUE	0	0x19 (25)	0xFFFF (-1)	0x19 (25)	0x8 (8)	0x1 (1)	0x10 (16)	0x0 (0)
MEMORY (ADDRESS)	0x0010 (16)							
VALUE	0x19(25)							

2.2.2 Code result

```
=====
CPU Instance 2: Halt on Misalignment Trap (program_halt.hex)
=====
Running program...
data memory address misalignment

----- Disassembled Program -----
0x0000: ADDI R1, R0, 25
0x0002: DIV R2, R1, R0
0x0004: REM R3, R1, R0
0x0006: JAL R4, 4
0x000a: SLTI R5, R0, 5
0x000c: ADDI R6, R0, 16
0x000e: SW R1, 0(R6)
0x0010: SW R1, 1(R6)

----- CPU Registers -----
R0 = 0x0000 (0)
R1 = 0x0019 (25)
R2 = 0xffff (65535)
R3 = 0x0019 (25)
R4 = 0x0008 (8)
R5 = 0x0001 (1)
R6 = 0x0010 (16)
R7 = 0x0000 (0)
PC = 0x0010
HALTED = true
```

```
----- Instruction Memory -----
address | instruction
0x00    | 0x8641
0x02    | 0x600a
0x04    | 0x700b
0x06    | 0xf104
0x08    | 0x88c5
0x0a    | 0xb145
0x0c    | 0x8406
0x0e    | 0xd031
0x10    | 0xd071
0x12    | 0x8007

----- Data Memory -----
address | data
0x0010  | 0x19
```